User's Manual for DDM Static/Fatigue Analysis of Laminated Composites v.1.0.0.

Ever J. Barbero

November 7, 2026

Abstract

This is a quick how-to manual on how to use the main.f90 *tool*, coupled with the DDM6v1.0.0-Gfortran *library*, for static and fatigue analysis of laminated composites under uniform strain, stress, or thermal loads. This tool is a one-Gauss-point solver, applicable to any symmetric laminated composite under uniform state of strain, thickness-averaged stress, or temperature. That is, applicable to uniaxial, biaxial, torsion, internal pressure, and any other in-plane uniform state of strain over the region of interest. A FEA implementation that can handle general problems is described in [1]. Static N=1 and fatigue N>1 analysis are provided for modes 0: thermal, 1:strain, and 2:stress. Furthermore, the tool showcased in this work has utility modes to help with the identification of relevant material properties using experimental data for both static and fatigue situations.

1 Introduction

Prediction of crack density is based on Discrete Damage Mechanics (DDM) described in [1, Ch. 9] and publications going back to [2]. For quasi-static loading, predictions hinge upon a single property, the mode-I fracture toughness G_{IC}. In turn, DDM enables for straightforward calculation of G_{IC} from a variety of experimental data that do not explicitly measure said material property.

The extension to fatigue analysis is also straightforward with the introduction of an additional property, the slope β of the defect nucleation function [1, Ch. 11]. All of this is valid for thermal-fatigue and constant-amplitude strain-driven fatigue tests.

For stress-driven fatigue tests, an additional property is needed because of the evolving laminate modulus E_x as a function of damage. Thus, the intercept a of the defect nucleation function $f(N)$ with the ordinate axis is needed [3]. Again, the material property a can be found from experimental data of crack density λ vs. number of cycles N or modulus reduction vs. N.

Note that we have used two terms: library and tool. The *library* is the supporting code that implements all the features of DDM and it can be called to predict crack density and other quantities at any x, y point in a laminate. The *tool* is the main.f90 code and a few auxiliary routines that, by calling the library, allows for a) identification of material

properties and b) simulation of the laminate in the domain $x, y, 0 < z < h$ on one x, y coordinate point in the laminate of total thickness $2h$.

2 The .inp file

We begin with the .inp file because it contains all that is needed, not only to predict fatigue damage but also for parameter identification that is a prerequisite for those predictions.

The nomenclature for all the input values is shown in Listing 1. The acronyms used are explained below. Note that each line may contain one or more data, depending on the problem at hand.

line 1 only one value, the number of laminas in half the laminate, which must be symmetric (a limitation of the theory that we are trying to overcome).

line 2 mono-ply thickness, usually 0.125 mm.[1] If two or more adjacent plies are laid out at the same angle, that number of plies per lamina is reported on line 4.

line 3 laminate stacking sequence reports the angles of all NL laminas. List NL values.

line 4 number of plies per lamina is reported here. List NL values. Anything typed after the required data is treated as a comment, in fact it is not even read by the tool.

lines 5, 6 fracture toughness mode I and II, 3 values each, are the coefficients of a quadratic polynomial to describe temperature-dependent properties. The first value is the constant terms and it is required. The other two can be zero, to indicate no temperature dependence. Then, 2 additional values each for the range of temperature for which said data was obtained experimentally. Outside the range, the properties are assumed to be constant at their value at the boundary of the range.

line 7 SFT is the stress-free temperature. TEmax–TEmin is the temperature range for experimental data supporting the elastic properties listed in lines 13–17. TTmax-TTmin is the temperature range for experimental data supporting the thermal properties listed in lines 18–19. If no temperature dependent data is available or needed, just enter zeros for everything except the first value, say E1_0, and the tool will assume the property is constant over the entire temperature range. You can enter TDP values for some properties and constant for others.

line 8 betaI, betaII are the mode I, II, slopes of the defect nucleation function f(N) that controls fatigue damage. The simulation will take place at exactly Ncycles, not in the range [1–Ncyles]. If you want a range, you have to perform several simulations, or leave a comment on GIT requesting such a feature.

lines9–12 Xt, Xc, are the longitudinal tensile and compression strengths of a lamina, Yc is the transverse tensile strength, and m is the Weibull modulus that describes the Gaussian distribution of fiber misalignment inside the composite [4, Ch. 8]. These are

[1] Any consistent system of units can be used but output formatting is optimized for MPa, mm, ^oC.

used to detect lamina failure, presently used internally but not reported by the tool, although reporting could be added upon request.

lines 13–17 are mechanical properties. See discussion on line 7 about the TDP range. Suffice to say that if you don't know the temperature dependency for some or all of these, just enter the first value, and zeros for the rest, to perform the analysis with some or all constant properties.

lines 18–19 are thermal properties. See discussion for line 7 about the TDP range. Suffice to say that if you don't know the temperature dependency for some or all of these, just enter the first value, and zeros for the rest, to perform the analysis with some or all constant properties.

line 20 The last item, labeled KEY, switches the tool to one of 5 modes. KEY=0: thermal, KEY=1: strain, KEY=2: stress, KEY=-1: GIc identification, KEY=-2: a, β identification.

KEY=0 The tool will read and use Tmin for a thermal analysis (static or fatigue depending on the value of Ncycles on line 8) in the range Tmin–SFT, and ignore the rest of the data on line 20.

KEY=1 The tool will read and use epsx, epsy, gammaxy for a strain-driven analysis in the range 0–epsx, 0–epsy, 0–gammaxy while incrementing all the strains proportionally. To avoid thermal effects, set Tmin=SFT.

KEY=2 The tool will read and use epsx, epsy, gammaxy as values for sigmax, sigmay, tauxy, and conduct a stress-driven analysis in the range 0–sigmax, 0–sigmay, 0–tauxy while incrementing all the stresses proportionally. To avoid thermal effects, set Tmin=SFT.

KEY=-1 used to identify GIc from experimental data $\lambda(\epsilon)$ or $\lambda(T)$. See § 3.4.

KEY-2 used to identify a, β from experimental data $\lambda(N)$ at ϵ or T constant. See § 3.5.

Listing 1: Input file (FILE5.inp). Line numbers not used in practice.

```
 1   NL
 2   MPT
 3   LSS
 4   n1          n2
 5   GIc0        GIc1    GIc2    TGImax  TGImin
 6   GIIc0       GIIc1   GIIc2   TGIImax TGIImin
 7   SFT         TEmax   TEmin   TTmax   TTmin
 8   betaI       betaII  Ncycles aI      aII
 9   Xt
10   Xc
11   Yc
12   m
13   E1_0        E1_1    E1_2
14   E2_0        E2_1    E2_2
15   G12_0       G12_1   G12_2
16   v12_0       v12_1   v12_2
17   v23_0       v23_1   v23_2
18   CTE1_0      CTE1_1  CTE1_2  CTE1_3
19   CTE2_0      CTE2_1  CTE2_2  CTE2_3
20   epsx   epsy    gammaxy     Tmin    KEY
```

Beware the software is evolving, with frequent new releases, that include this document. Check out the repository at `https://github.com/everbarbero/DDM_release` .

Listing 2: Example FILE5.inp

```
    2       comments like these are not read by main.exe
0.125                                                           mpt
0           90                                                  LSS
2.00        4.00                                                np
0.158       0.00        0.00        1.000E+03  -1.000E+03       GIc
0.48        0.00        0.00        1.000E+03  -1.000E+03       GIIc
150.        1.000E+03  -1.000E+03    1.000E+03  -1.000E+03      SFT
0.00        0.00        1.00         0.00        0.00           b1,b2,N,a1,a2
2274.                                            Xt
1849.                                            Xc
255.                                             Yc
8.90                                             m
4.470E+04   0.00        0.00                   E1 3 coef
1.270E+04   0.00        0.00                   E2
5.800E+03   0.00        0.00                   G12
0.297       0.00        0.00                   v12
0.410       0.00        0.00                   v23
3.70        0.00        0.00         0.00      CTE1 4 coef
30.0        0.00        0.00         0.00      CTE2
0.00480     0.00        0.00       150.00  -1   0:thermal,1:strain,2:stress
=== this line and below, not read, add your own comments if you wish ===
```

3 Basic Usage

Listing 3: Distribution files

```
main.f90
ddm6.a                    64-bit x86 static library
libblas.a                 64-bit x86 static library
liblapack.a               64-bit x86 static library
FILE5.inp
epsxLambdaExperim.txt     2-column experimental data
NcylesLambdaFit.txt       2-column experimental data
1mmpermin.txt             experimental data
plot_2020.py              lambda-epsx
plot_2022.py              Ncycles-lambda
NcyclesGIcPrime.py        Plot the f(N) function
```

3.1 Thermal Fatigue: KEY=0

A change of temperature produces thermal strain, so thermal fatigue is a particular form of mechanical fatigue where strain appears in every direction.

The Discrete Damage Mechanics (DDM) code was originally developed for static strain loading. Why strain, not stress? Because under load, all laminas are subjected to the same strain, not the same stress, and within each lamina, all fibers and the matrix are subjected to the same strain, not stress.

Subsequently, prediction of fatigue behavior was initially developed for thermal fatigue, because thermal testing is so slow that testing for 500 cycles is scarce and more than 4000 cycles prohibitive [5]. Compare to mechanical fatigue testing where a million cycles is customary. Thus, a predictive tool for thermal fatigue is indispensable. A method to simulate thermal fatigue testing using much faster surrogate strain fatigue testing is proposed in [6, Ch. 6].

For thermal fatigue there are two issues: fatigue and temperature dependent properties (TDP). The library and the tool both accommodate TDP in a way that if you don't have the data you can use the tool with constant properties by providing zero values for the TDP component of the properties.

There are two types of properties: mechanical and thermal. Both can be temperature dependent. Please refer to Listing 1 lines 13–19.

Mechanical properties, moduli (E1,E2,G12) and Poisson ratios (v12,v23) are represented with quadratic polynomials (3 coefficients). The first coefficient is mandatory, representing the constant property, and the other two can be zero.

The coefficients of thermal expansion (CTE1, CTE2) are thermal properties, represented with cubic polynomials (3 coefficients) because that is necessary to fit the available data [1, Ch. 11]. The first coefficient is mandatory, representing the constant property, and the other three can be zero.

The strength properties (lines 9–12 in Listing 1) are used to predict failure modes in 1–longitudinal tension (lines 9 and 12) [4, §8.2], 2–longitudinal compression (line 10) [4, §8.3], and transverse compression (line 11) not implemented yet.

For thermal load, you must impose $SFT > Tmin$, so that always $\Delta T = Tmin - SFT < 0$ because crack density only increases under tensile strain, that is under temperature induced contraction. For strain- or stress-loading you must impose $Tmin = SFT$. Simultaneous mechanical and thermal load is not implemented. You may leave a request on GITHUB it you need such a feature.

The *tool* output is shown in both the console and FILE7.out. It reports columns with the temperature from SFT down to Tmin, the corresponding thermal strain, and the crack density for all columns. 0.02 that is the seed crack density needed by DDM to define the representative volume element, and from it see if the crack density increases or not. Think of it as the initial density of defects naturally present in the material.

3.2 Strain-driven Fatigue: KEY=1

In a displacement-controlled mechanical test, strain is applied in one direction, unless the setup is biaxial or the fixture applies only shear. All these cases can be accommodated in line 20 of the .inp file.

Whether the simulation is static or fatigue strain is controlled by the value of Ncycles on line 8, col. 3.

For strain- or stress-loading you must impose $Tmin = SFT$. Simultaneous mechanical and thermal load is not implemented. You may leave a request on GITHUB it you need such a feature.

The output is shown in both the console and FILE7.out. It reports columns with the derived stress, the driving strain, and the crack density for all columns.

3.3 Stress-driven Fatigue: KEY=2

3.4 Identification fracture toughness GIc: KEY=-1

Start with a table with crack density λ vs. strain ϵ_x data in file `epsxLambdaExperim.txt`. If instead you have experimental data of laminate modulus E_x vs. strain ϵ_x, the tool does not support it yet, but it could. You may leave a request on GitHub if you need that, and be sure to attach a sample of the experimental data (two columns, text file).

1. KEY=-1. In `FILE5.inp` use `KEY=-1` to identify[2] the fracture toughness GIc. For KEY=-1, the .inp file needs.
   - line 20, col 1: epsx strain found on on the last row in `epsxLambdaExperim.txt`, i.e., experimental data with columns: epsx, lambda.
   - line 5, col 1: GIc, user-set upper-limit for the search, larger than expected but not too large.
2. Run *the tool* (`main.exe`). GIc appears on the console and on FILE8.out
3. Now, use the new value of GIc to update your .inp file.

[2] i.e., identify=calculate from data

4. KEY=1. Run with `KEY=1` to calculate $\lambda(\epsilon_x)$ in the range $0 < \epsilon_x < \epsilon_x^{max}$, with ϵ_x^{max} given in line 20, col 1 of the .inp file.

5. `FILE7.out` contains the results, including: stress, strain, and λ for laminas i=1..NL, with the number of laminas NL given on line 1 of the .inp file.

6. `firstCrakAtStrain.txt` is used by the code to inform further analysis options. DO NOT DELETE it. It contains epsx and lambda values as soon as the first cracks appears. You can verify this by comparing to FILE7.out; look for 0.2000E-01 changing to a higher value and confirm the firstCrackAtStrain value is correct.

7. Now you can use `plot_2020.py` to plot FILE7.out on top of the experimental data from `1mmpermin.txt` provided in the distribution. Be sure you *just now* ran the tool with KEY=1 because the name FILE7.out may have been used before and it is also used later for tool option KEY-2.

3.5 Identification of fatigue parameters a, β

1. Before you start, either you know the value of GIc or run KEY=-1 to find it. Either way, you must now run KEY=1 to set up files needed by KEY=-2. **Don't run KEY=-2 without previously running KEY=1.**

2. Now use `KEY=-2` in the same `FILE5.inp` that you have been using.

3. You need a file `NcylesLambdaFit.txt` with two columns: `Ncycles` and `lambda`. It could be straight from experiments, or the output from a model equation (quadratic, etc.) used to smooth out the experimental data. Python code to smooth out experimental data is included in `plot_2022.py`.

4. As soon as you run the tool, `NcylesLambdaFit.txt` is copied into `NcyclesGIcPrime.txt` and two additional columns are added to report the calculated fatigue fracture toughness G'_{IC} and defect nucleation function $f(N) = G'_{IC}/G_{IC}$. That's all you need to plot the defect nucleation function $f(N) = a + \beta \log_{10} N$, as shown in [1, Fig. 11.4].

5. The tool calculates the fatigue parameters a, β and appends them to FILE8.out

6. The code `NcyclesGIcPrime.py` can be used to plot the f(N) diagram.

References

[1] Ever J. Barbero. *Finite Element Analysis of Composite Materials Using Abaqus, Second Edition*. CRC Press, Boca Raton, FL, 2023.

[2] Ever J. Barbero and D. H. Cortes. A mechanistic model for transverse damage initiation, evolution, and stiffness reduction in laminated composites. *Composites Part B*, 41:124–132, (2010).

[3] Nikolay V. Turbin, Kirill A. Shelkov, and Nikolay O. Kononov. Cyclic damage quantification in composite materials using discrete damage mechanics. *Composite Structures*, 342, (2024)118271.

[4] Ever J. Barbero. *Introduction to Composite Materials Design, Third Edition*. CRC Press, Boca Raton, FL, 2018.

[5] Ever J. Barbero and J. Cabrera. Master paris law for thermal fatigue of laminated composites. *J. Composite Materials*, 52(30):1–18, (2020).

[6] Javier Cabrera Barbero. *Thermal-Fatigue and Thermo-Mechanical Equivalence for Transverse Cracking Evolution in Laminated Composites*. PhD thesis, West Virginia University, 2018.